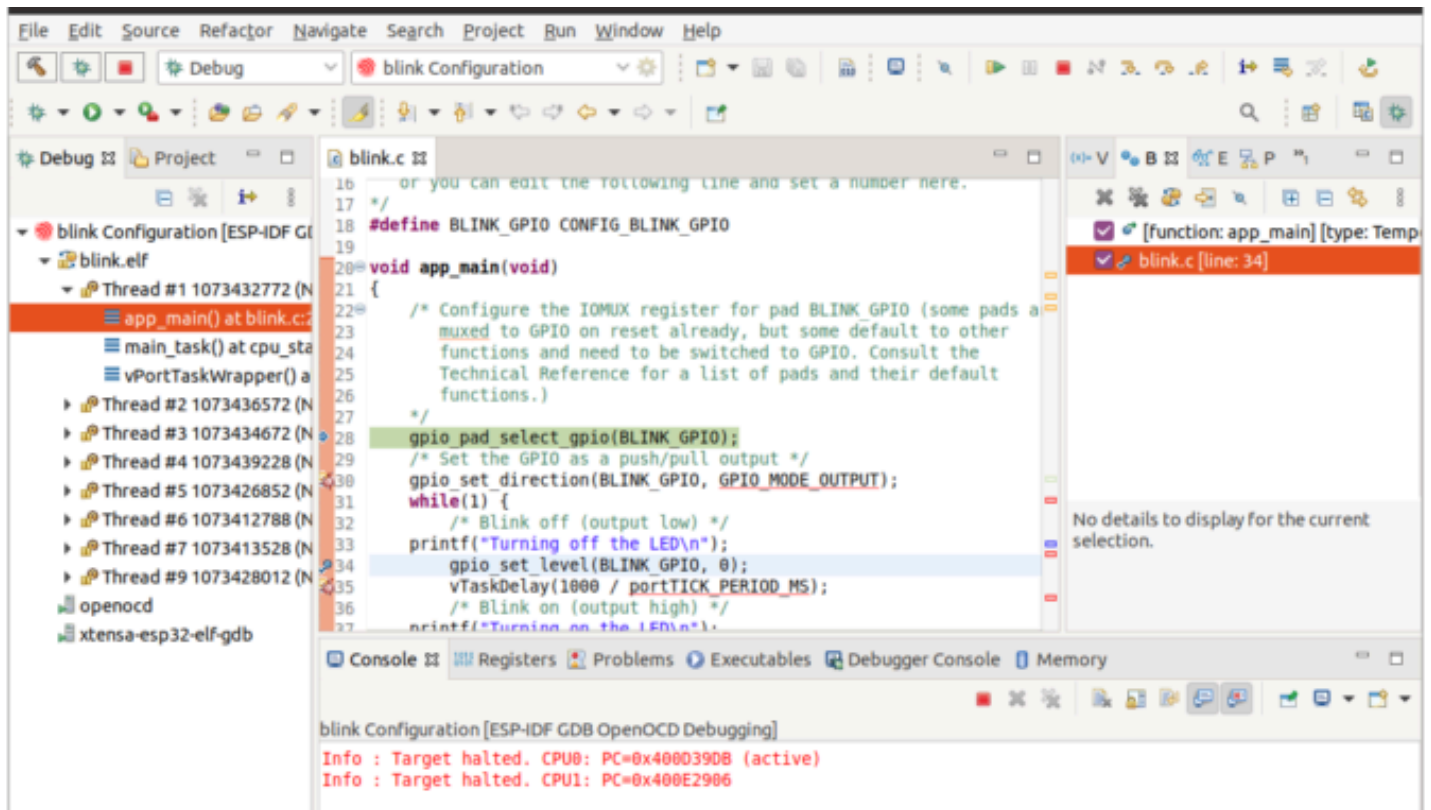


Espressif esp-idf & Eclipse IDE: Debugging with JTAG and OpenOCD



Developing ESP32 code by iteratively downloading code to your target via esptool can become laborious for anything but the smallest of projects.

In addition to the UART Bootloader, the ESP32 can also be programmed and debugged via the JTAG interface. Debugging allows single stepping through your code, adding breakpoints and examining the contents of registers.

Besides the rich debugging, programming over JTAG can also speed up development as the JTAG interface can be as fast as 20MHz as opposed to the serial download at a typical 460,800 bps.

Setup

This tutorial assumes you have Eclipse and the esp-idf set-up on Ubuntu 20.04.01 LTS. If not, please visit [Getting started with the Espressif esp-idf & Eclipse IDE](#)

It also assumes you are using an FTDI based JTAG interface such as the Espressif [ESP-Prog](#).

Running OpenOCD as a normal user will probably result with the following error message:

```
1. Error: libusb_open() failed with LIBUSB_ERROR_ACCESS
```

To prevent this occurring, create the file `/etc/udev/rules.d/99-openocd.rules` and add the following contents:

```
1. # Copy this file to /etc/udev/rules.d/
2.
3. ACTION!="add|change", GOTO="openocd_rules_end"
4. SUBSYSTEM!="usb|tty|hidraw", GOTO="openocd_rules_end"
5.
6. # Please keep this list sorted by VID:PID
7.
8. # Original FT232/FT245 VID:PID
9. ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6001", MODE="664", GROUP="plugdev"
10.
11. # Original FT2232 VID:PID
12. ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6010", MODE="664", GROUP="plugdev"
13.
14. # Original FT4232 VID:PID
15. ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6011", MODE="664", GROUP="plugdev"
16.
17. # Original FT232H VID:PID
18. ATTRS{idVendor}=="0403", ATTRS{idProduct}=="6014", MODE="664", GROUP="plugdev"
19.
20. LABEL="openocd_rules_end"
```

Then restart udev using:

```
1. #sudo udevadm trigger
```

You will also want to be a member of the plugdev group. By default, Ubuntu users are normally members of the plugdev group and hence no action is required.

Update OpenOCD-esp32

OpenOCD-esp32 should already be installed in your home directory at `~/.espressif/tools/openocd-esp32/`

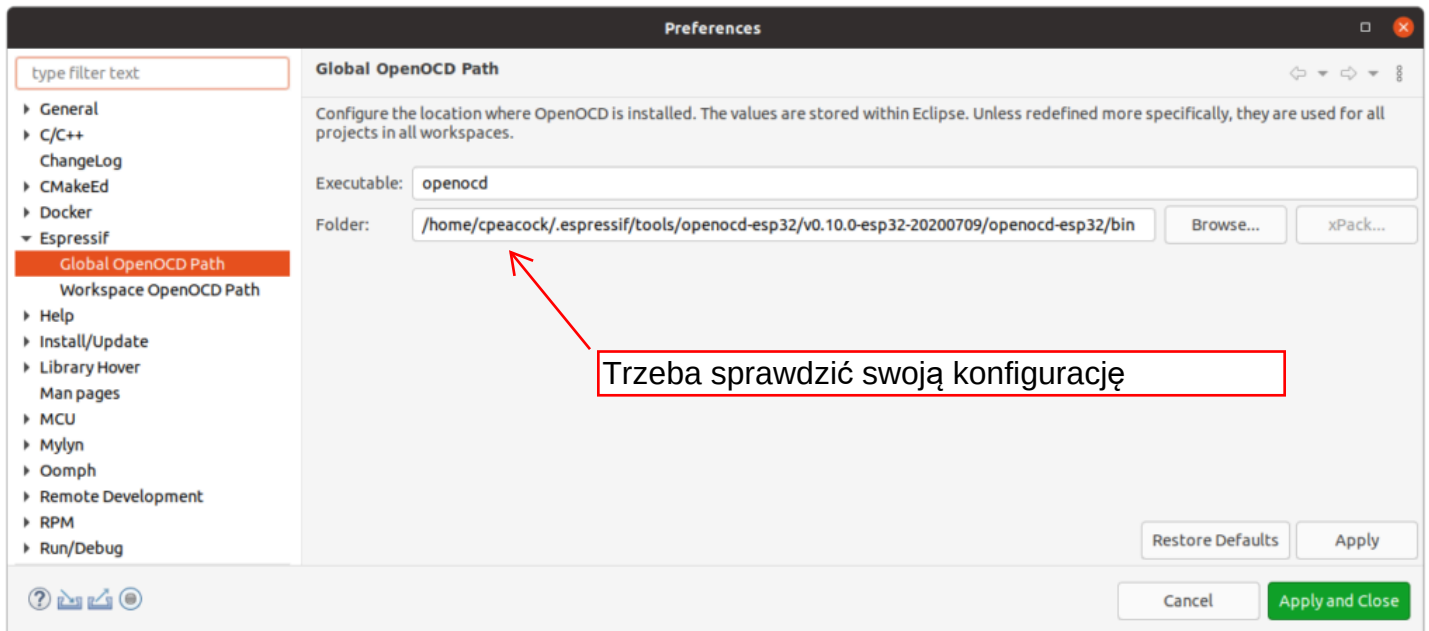
However we are noticing regular updates and bug fixes with OpenOCD-esp32. Release information detailing features and bug fixes can be found at <https://github.com/espressif/openocd-esp32/releases>

If you don't have the latest version of OpenOCD, download and extract a copy:

```
1. cd ~/.espressif/tools/openocd-esp32/
2. mkdir v0.10.0-esp32-20200709
3. cd v0.10.0-esp32-20200709
4. wget https://github.com/espressif/openocd-esp32/releases/download/v0.10.0-esp32-20200709/c
5. tar -xzf openocd-esp32-linux64-0.10.0-esp32-20200709.tar.gz
```

Once downloaded, the new version needs to be specified in Eclipse. Within Eclipse, from the *Window* Menu, select *Preferences*:

In the left hand panel, under *Espressif*, select *Global OpenOCD Path*:



Change the path to reflect your new version and click *Apply and Close*.

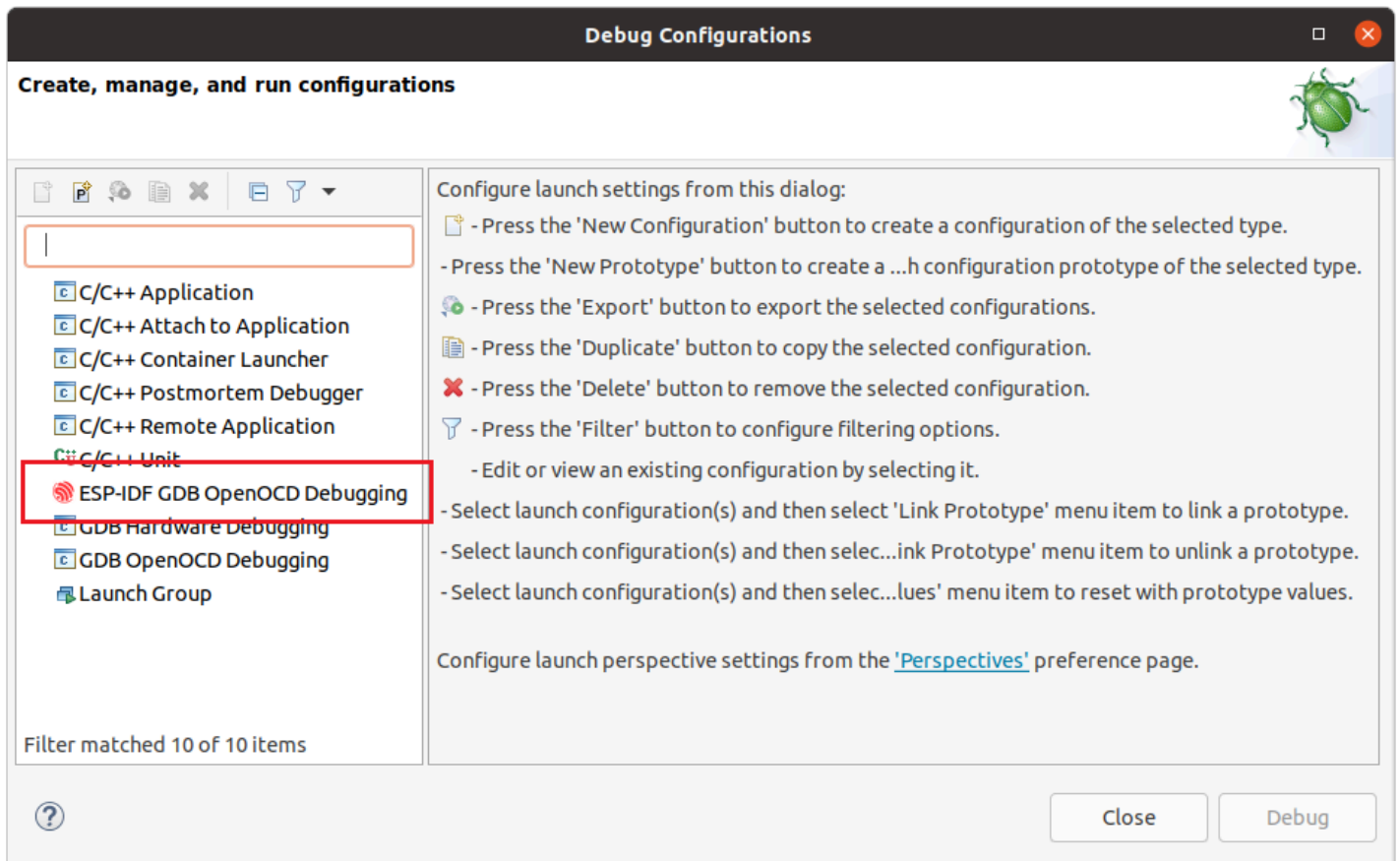
Debugging Configurations in Eclipse

First, open/**import** an ESP-IDF project. For this example we use
esp-idf/examples/get-started/blink

(If no project is open, you may get a java.lang.NullPointerException error.)

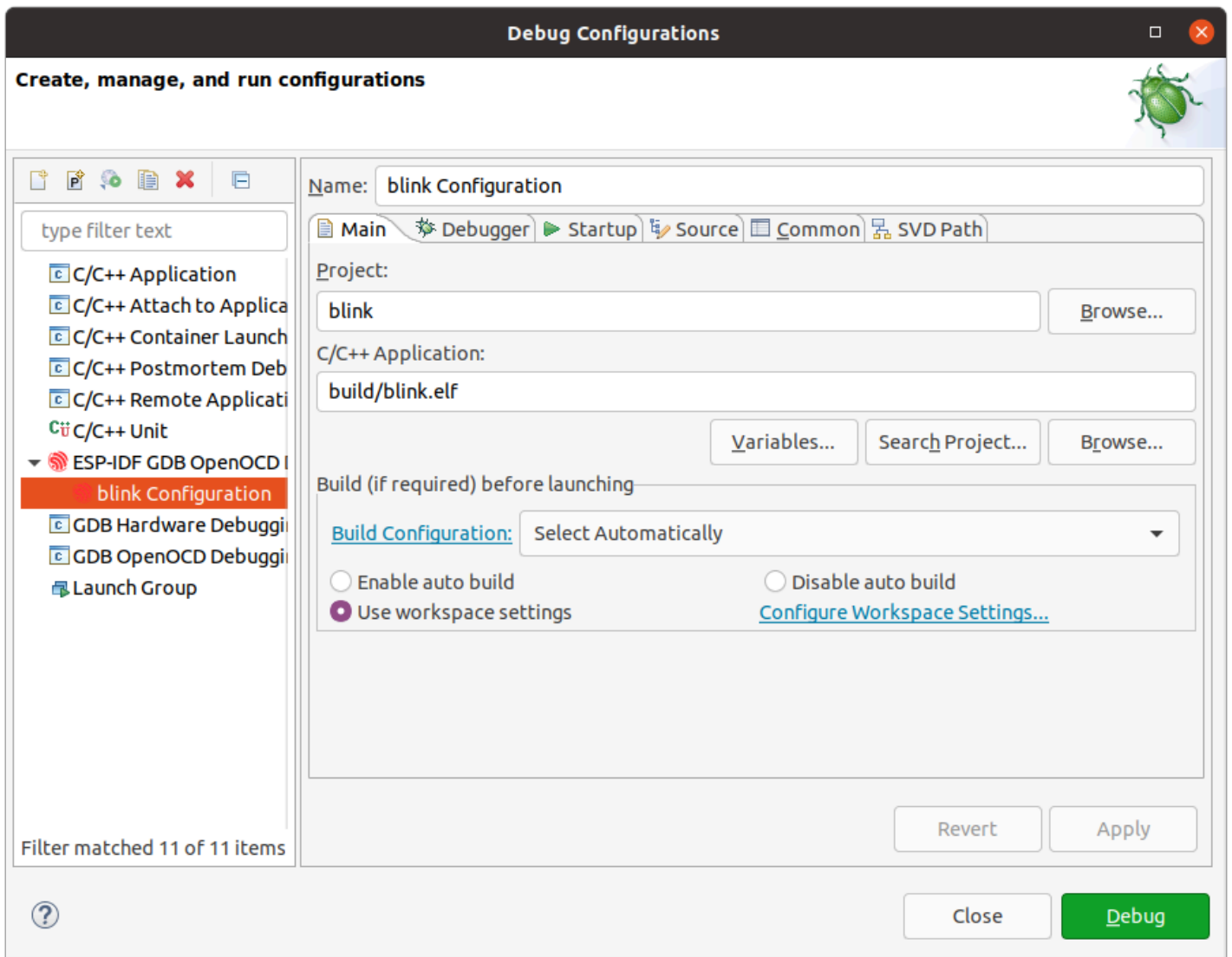
Second, **build** your project so a binary file exists in the build directory.

From the *run* menu, select *debug configurations*

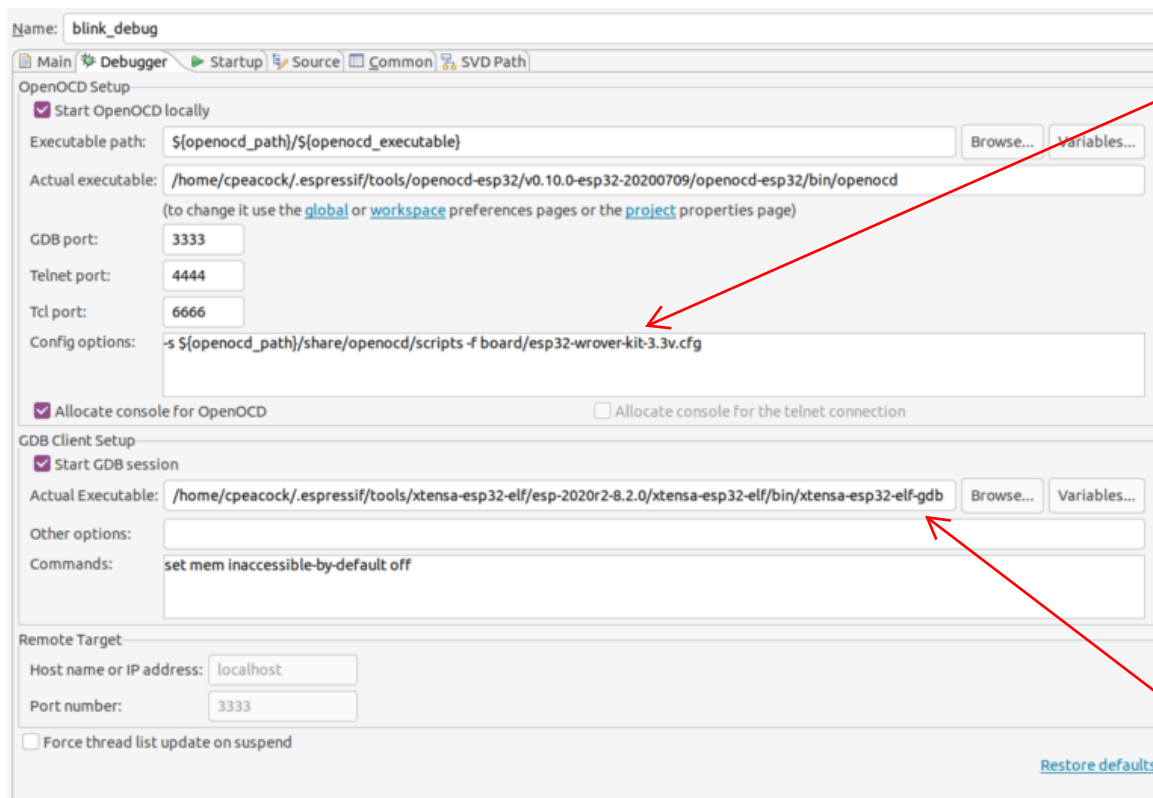


Right click on *ESP-IDF GDB OpenOCD Debugging* and select *New Configuration*.

The settings on the *Main* tab should look like below and should not need amending.



Next, click on the *Debugger* Tab:



Trzeba wpisać swoją konfigurację. Ścieżka do scripts powinna być też. Najlepiej sprawdzić wywołując openocd.exe w terminalu z konfiguracją. U mnie działają opcje openOCD podstawowe:

```
-s C:\Espressif\tools\openocd-esp32\v0.12.0-esp32-20250707\openocd-esp32\share\openocd\scripts -f interface/ftdi/esp_ftdi.cfg -f target/esp32.cfg -c "adapter speed 400"
```

można też dodać:

```
-c "init" -c "reset halt" -c "esp32.cpu0 configure -rtos FreeRTOS" -c "esp32.cpu1 configure -rtos FreeRTOS" -c "set ESP_RTOS none"
```

Zwiększenie prędkości do:

```
-c "adapter speed 2000" <----- też działa
```

Ścieżkę GDB też trzeba sprawdzić.

Change the Config options to

```
1. -s ${openocd_path}/share/openocd/scripts -f interface/ftdi/esp32_devkitj_v1.cfg -f board/e
```

Now click on the *Startup* Tab:

The screenshot shows the Eclipse IDE's 'Startup' tab for a debug configuration named 'blink_debug'. The configuration is set up for an ARM target using OpenOCD. Key settings include:

- Initialization Commands:**
 - ☒ Initial Reset. Type: `init`
 - Commands: `mon reset halt`, `flushregs`, `set remote hardware-watchpoint-limit 2`
 - ☒ Enable ARM semihosting
- Load Symbols and Executable:**
 - ☒ Load symbols
 - ☒ Use project binary: `blink.elf`
 - ☐ Use file: [text field] [Workspace...] [File System...]
 - Symbols offset (hex): [text field]
 - ☐ Load executable
 - ☒ Use project binary: `blink.elf`
 - ☐ Use file: [text field] [Workspace...] [File System...]
 - Executable offset (hex): [text field]
- Runtime Options:**
 - ☐ Debug in RAM
- Run/Restart Commands:**
 - ☐ Pre-run/Restart reset. Type: `halt` (always executed at Restart)
 - [text field]
 - ☐ Set program counter at (hex): [text field]
 - ☒ Set breakpoint at: `app_main`
 - ☒ Continue

A 'Restore defaults' link is visible at the bottom right of the configuration window.

The default debugging configuration assumes you have already downloaded your source code to the target and you are attaching the debugger to a programmed target.

If you want to download the code first, which is typical in a iterative programming development cycle, add the following to the top of you Initialization Commands:

```
mon program_esp ${workspace_loc:blink/build/blink.bin} 0x10000 verify
```

This will download your code over the JTAG link prior to establishing the debugging session.

Debugging

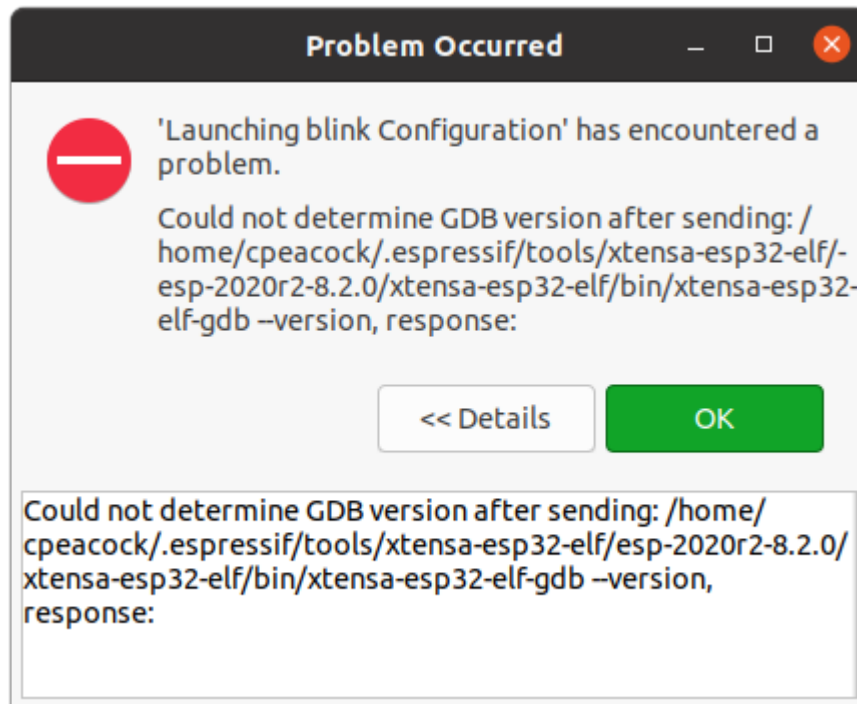
Espressif has a great document on debugging within Eclipse: <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-guides/jtag-debugging/debugging-examples.html>

Potential Errors

Below are some potential errors you may face with openocd and esp32-gdb.

xtensa-esp32-elf-gdb requires shared library libpython2.7.so.1.0

Error: *Could not determine GDB version after sending: .../esp-2020r2-8.2.0/xtensa-esp32-elf/bin/xtensa-esp32-elf-gdb -version, response:*



If you receive this error and try to run xtensa-esp32-elf-gdb on the command line, you are likely to get the following error suggesting a library is missing:

```
1. xtensa-esp32-elf-gdb: error while loading shared libraries: libpython2.7.so.1.0: cannot op
```

GDB (8.1.0.20180627) is currently built with Python 2.7 libraries. You will need to manually install these:

```
1. sudo apt-get install libpython2.7-dev
```

Espressif developers indicate they plan to switch to building GDB with Python 3 libraries in the next toolchain release.

When all else fails...

Espressif has comprehensive documentation at [JTAG Debugging](https://www.beyondlogic.org/debugging-with-jtag/)